CSC120 2026S Weeks 15 & 16 Lab No.12: Connect Four

1 Introduction

The project aims to develop a program for the game Connect Four, a game like Tic-Tac-Toe with two players, with the following features:

> The game is played on an upright board with seven columns and six rows. The players take turns inserting their pieces into a column. A column can accept a piece if it has fewer than six pieces. In each column, a piece is inserted flush to the bottom. A player must form a line with four pieces to win the game. A line can be horizontal, vertical, diagonal, or anti-diagonal.

Figure 1 presents an example of a board configuration, where `'O'` represents Player 1, and `'X'` represents Player 2.

	5							
	4							O
	3				O			O
row positions	2	O	O	X	X			X
	1	X	O	O	X			O
	0	O	**X**	**X**	**X**	**X**	O	X
		0	1	2	3	4	5	6
		column positions						

Figure 1: In this configuration, Player 2 has won.

Your program will consist of an object class `Board` and a main class `Main`. Templates for the two classes are available. A video here shows sample executions.

2 Static Constants

The templates include some static constants that can be useful when writing code. They define the constants `LENGTH`, `ROWNUM`, and `COLNUM` with values 4, 6, and 7, respectively. Using these constants in your code is *not required*, but it is recommended to avoid accidentally using the wrong literal, e.g., 8 instead of f6. In addition, they define `char` constants for open space, Player 1, and Player 2. They are `CHAR_OPEN = ' '`, `CHAR_P1 = 'O'`, and `CHAR_P2 = 'X'`, respectively. These have indices 0, 1, and 2 and are stored in a 3-element array `CHAR_LIST`.

We aim to color-code the pieces (e.g., red and blue). The class `Board` has three static `String` constants that specify any open space, the Player 1 piece, and the Player 2 piece. They are `STRING_OPEN`, `STRING_P1`, and `STRINGP2`, respectively. The three `String` values are put into a three-element array `STRING_LIST`.

`STRING_OPEN` is simply one whitespace. The other two are colored `"O"` and `"X"`. respectively. You can use an escape sequence to specify the foreground (font) color, the background color, and to cancel coloring. There are 256 colors from 0x00 to 0xFF. Four constants are defined in the template for `Board`: `P1_FORE_COLOR`, `P2_FORE_COLOR`, `P1_BACK_COLOR`, and `P2_BACK_COLOR`, where

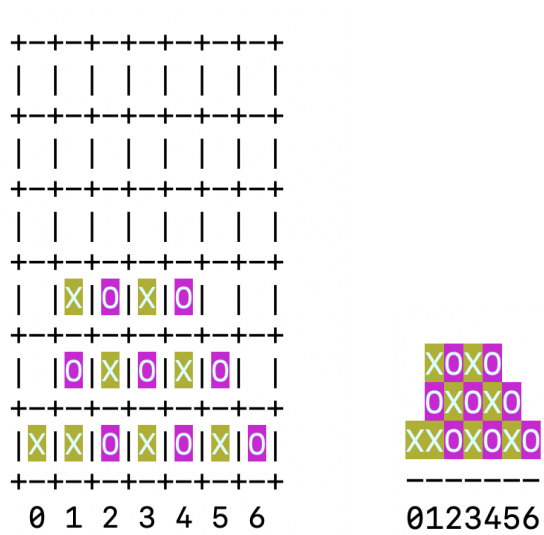

Figure 2: The left one has frames, and the right one does not.

"FORE" refers to the foreground and "BACK" refers to the background. The `String` constant specifications use `String.format` with the color substitutes for integer placeholders.

The board presentation with the color selections in `Const` can be as follows: You may choose different color combinations in your code. `SelectColor` is a program made available for exploring color combinations. .

3 Class `Board`

`Board` is an object class representing the board. A straightforward implementation of a Connect Four board is to use (a) a two-dimensional rectangular `int` array for maintaining the pieces on the board column-wise from bottom to top, (b) a one-dimensional `int`array for maintaining the height value of each column, and (c) an `int` variable representing the total number of pieces in the board. Thus, the dimensions of (a) are `COLNUM` by `ROWNUM`, and the dimension of (b) is `COLNUM`. Let's call the three instance variables `table`, `height`, and `size`, respectively. The board configuration in Figure 1 is represented by: `height`: `{3,3,3,4,1,1,5}`, `size`: `20`, and `table`: `{{1,2,1,0,0,0},{2,1,1,0,0,0},{2,1,2,0,0,0},{2,2,2,1,0,0},{2,0,0,0,0,0},`
`{1,0,0,0,0,0},{2,1,2,1,1,,0}}`

The methods required for this class are as follows, where `col`, `row`, and `player` represent the column index (from 0 to `COLNUM-1`), the row index (from 0 to `ROWNUM-1`), and the player index (1 or 2), respectively:

- The Constructor: This assigns `table` and `height` to their respective new array objects. Since `0` represents the open location and "zero-height," the default assignments of `0` to the array elements work for initialization. The variable `size` must be initialized with the value of `0`.

- `Board clone()`: This method returns a clone of the object. You can implement this by calling the constructor to create a new `Board` object, say +verb+cloned+, copying everything to `cloned`, and returning `cloned`.
- `int getPiece( int col, int row )`: This method returns the piece at the coordinate. If the coordinate is valid, return the element from `table` corresponding to the coordinate. Otherwise, the method returns `-1`.
- `int getHeight( int col )`: This method returns the element from `height` corresponding to the column index.
- `boolean isOpen( int col )`: This method returns if the element from `height` corresponding to the column is less than the height of the board (i.e., `ROWNUM`).
- `int getTop( int col )`: This method returns `-1` if the height of the column is 0. Otherwise, since the height specifies the next element's row position, the method returns the element immediately below it.
- `boolean isAllFull()`: This returns if `size` is equal to `COLNUM * ROWNUM`.
- `boolean add( int col, int player)`: This method returns `false` if the column is full. Otherwise, the method places the piece at the coordinate of the next piece in the column (the row position should be `height`), increases verb+height+ by 1, increases `size` by 1, and returns `true`.
- `boolean isWinner( int player )`: This examines all line segments consisting of `LENGTH` consecutive slots in the `table`. If any segment has all `player`, the player has won. A straightforward way to write this method is to use four separate sections for vertical, horizontal, diagonal, and anti-diagonal searches. Each search can be implemented using triple loops. The outer two loops will iterate over all possibilities for one end of a segment, and the innermost loop will iterate from 1 to `LENGTH` to move the coordinates for checking. You can take advantage of `getPiece`'s property that it returns `-1` if the coordinates are invalid.

4 Class `Main`

Once you complete writing `Board`, you can move on to writing the game program `Main`. The template provides the main method. The main method calls a static void method that plays one game with the user as Player 1 and the computer as Player 2. Since `oneGame` may be long, you may use a static `Board` variable so you do not have to pass the board between methods.

`oneGame()` initializes a static `BoardInt` variable, assigns it to a `Board` object, prints the intelligence level information, and receives the level. The prompt for receiving the level can be as follows:

```
Let's play Connect Four.
Pick the intelligence level
0: You play the opponent too
1: The program plays randomly
2: The program does not miss a winning play
```

```
3: The program additionally avoids a losing play
Enter your choice >
```

You may want to convert unexpected user input to a fixed value, such as 1.

After printing the board (which has no pieces) on the screen, `oneGame` enters a loop, which continues to run while the board is not full and neither player has won:

1. If the board is full, report that the game is a draw and terminate the loop.

2. Ask the user to enter a column number (`0..COLNUM-1`) and receive a number. You may want to convert unexpected user input to an open column or repeat the interaction until the user enters a valid open column number. Add Player 1's piece to the column and print the board. If Player 1 wins, report this and terminate the loop.

3. If the level is 0, ask the user to enter the column. A similar adjustment to the previous one should be made. Otherwise, if the level is 1, select an open column. Otherwise, use some logic to select a column. Add Player 2's piece to the column chosen and print the board. If Player 2 wins, report this and terminate the loop.

5 Logical Column Selection

For Levels 2 and 3 intelligence, you need to be able to see what happens if a piece is inserted in a column.

For Level 2, let P be 1 or 2 (i.e., a player number). Whether or not Player P has a winning play is determined by checking if the player can win by playing the column `j` for some `j`. The question about whether Player P can win by playing `j` can be tested by creating a clone of the board, inserting Player P's piece in the column, and checking if Player P has won.

For level 3, whether or not Player P has a losing play is determined by checking if the opponent can win by playing the column `j` for some `j`. The question of whether the opponent can win by playing `j` can be tested by creating a board clone, inserting her piece into the column, and checking whether she has won.